

Playbook

How the Enterprise Task Monitoring System works, who it is for, and the rules it will not bend.

SYSTEM	Task monitoring across four departments, three roles, on a cadence each department sets
BACKEND	Node · Express · Prisma · MySQL 8
FRONTEND	React · Vite · Material UI (JavaScript, not TypeScript)
AI	Google Gemini <code>gemini-2.5-flash</code> , opt-in per department
COMPANION	Module Flowcharts (separate PDF) — one diagram per module
GENERATED	21 July 2026

This playbook describes behaviour that is implemented and running, not intentions. Where the system refuses to do something — delete a department, cancel work already begun, show an employee the AI's assessment of them — the refusal is documented as a rule, because that is what it is.

1 What this system is

A record of what every employee did, block by block, that managers can trust and employees can live with.

Ara Workbench asks each employee one question per block of working time — hourly by default, or every two or three hours where that is the honest cadence: *what did you complete?* Those answers roll up three ways off a single table — by person, by project, by department — so progress can be reported without anyone maintaining a second set of books.

Around that core sit four things: **assignments** (work handed out, so hours can be checked against intent), **approvals** (a Tech Lead signs off each day), **analytics** (a nightly rollup feeding the dashboards), and an **AI analyser** that asks, on a cadence Management sets, whether the hours logged actually advance the work assigned.

The design position

Two decisions shape everything else, and both are unusual enough to state plainly.

AN ENTRY RECORDS WORK ALREADY DONE

A task entry therefore has **no status and no priority**. Every row is, by definition, completed work. A column that can only ever hold one value is not information — it is a field that invites people to lie to it, and a denominator that quietly inflates. Status lives on the *day* (an approval state) and on *assignments* and *project modules*, which are genuinely work still ahead.

THE WORK IS THE COMPANY'S, NOT THE PERSON'S

Removing a person, a team, a department or an hour must never remove the record of work that was done. Deleting an employee removes their *login*; their logged hours stay, still attributed to them by name. This was once a cascade, and deleting one employee silently erased months of delivery history.

Who uses it

ROLE	WHAT THEY DO HERE	WHAT THEY CAN SEE
Employee	Logs each working hour, submits the day, works assignments	Their own rows only
Tech Lead	Approves or returns sheets, assigns work, exports reports, reads AI findings	Their whole department
Management	Administers departments, people, teams, projects, settings; reads the audit log	The whole company

Departments are **data, not an enum**. Adding a fifth is a row, not a release: its working hours, its grid columns and any extra form fields are all configuration. The four seeded departments are Tech Team, Digital Marketing, Social Media Management and Video Editing.

ISOLATION

A Tech Lead of Video Editing must never, under any code path, see a Digital Marketing employee's task. This is not a check that each endpoint remembers to make — it is a filter that is impossible to omit. See §4.

2 Running it

From a clean machine to a signed-in browser.

Prerequisites

- Node.js 20+ and npm
- MySQL 8 — `docker-compose.yml` at the repo root brings one up on port **3307**, deliberately off 3306 so it cannot clash with a MySQL you already have

First run

#	STEP	COMMAND
1	Database	<code>docker compose up -d</code>
2	Backend config	copy <code>backend/.env.example</code> → <code>backend/.env</code> , fill it in
3	Install	<code>npm install</code> in <code>backend/</code> and <code>frontend/</code>
4	Schema	<code>npm run db:deploy</code> (production) or <code>db:migrate</code> (dev)
5	Seed	<code>npm run db:seed</code>
6	API	<code>npm run dev</code> → <code>http://localhost:4000</code>
7	Web	<code>npm run dev</code> in <code>frontend/</code> → <code>http://localhost:5173</code>

CONFIG IS VALIDATED AT BOOT

Every variable in `.env` is checked by `src/config/env.js` when the process starts. A bad config **refuses to start** rather than failing three hours later inside a request handler. If the API will not boot, read the first lines of its output — it names the variable.

Health and prefixes

Liveness is `GET /healthz`. Every API route sits under `/api/v1`. There is no `/api/health`; looking for one is the usual first wrong turn.

Seeded accounts

The seed creates one Management account plus a populated demo company. Every seeded user shares the password `Password@2026!`; the Management account's password is set from your environment. All seeded users are flagged `mustChangePassword`, so the first sign-in goes straight to a forced password change — which is also the fastest way to confirm the auth chain works end to end.

Settings that are not in .env

Business policy lives in **Admin > Settings** and takes effect without a redeploy: reminder grace minutes, auto-approval hours, the backdating window, autosave debounce, whether daily submission is required, and how often the AI analyser runs. Secrets and infrastructure — the database URL, the JWT secret, the SMTP password, the Gemini key — are environment variables only, and are never rendered or returned by any endpoint. An admin UI that can rewrite the JWT secret is an admin UI that can compromise the whole system.

3 A day in the system

The same day, from three seats.

The employee

1. Opens **My Task Sheet**. The grid's columns come from their department's configuration — Tech sees 10:00–18:00, Video Editing sees its own shift, from the same screen.
2. Each block, fills a cell: *what did you complete?* plus a project. If they have open assignments, a third question appears first — **which task?** — with an honest "Other work" option.
3. Text autosaves after 1.2 seconds. A small indicator says saving / saved / unsaved — never a blocking spinner.
4. Logging an hour late is **allowed** and flagged **LATE**. The system measures the behaviour rather than preventing the record.
5. When every required hour is filled, **Submit** becomes available. Until then its tooltip says how many hours are outstanding.

The Tech Lead

1. **Approvals** shows submitted sheets **oldest first**, with filled/expected hours and how long each has been waiting.
2. **Review** deep-links to **Monitor** — the person's actual hourly sheet — because approving a number without reading the work is not review.
3. Approve, or Return with a note. The note is a required field in the dialog, not a 422 after the fact.
4. A lead may correct an entry directly. It is badged **LEAD EDIT** in the grid and recorded against the lead's name in the revision history. A lead's edit is never silent.
5. **AI Insights** lists findings from the analyser, each expandable to the exact evidence the model was given.

Management

1. **Dashboard** opens on **today**, not a 30-day range: per-department compliance and a list of names to chase. Everything else is behind "Detailed analytics".
2. Administration covers departments, employees, teams, projects, the audit log and settings.
3. Management has no department and therefore no task sheet — they do not log hours, and the nav hides it.

TWO THRESHOLDS, DELIBERATELY DIFFERENT

Two hours past an unfilled slot earns a private nudge. Three hours puts your name on the Management dashboard. The gap exists so that a nudge stays a nudge.

What happens without anyone

Nine scheduled jobs keep the system honest when nobody is looking: hourly reminders with a three-way escalation, a lead digest twice a day, a management summary at 19:00, auto-approval of sheets nobody reviewed within 24 hours, the nightly rollup, the 180-day retention sweep, and the AI analyser. See §8.

4 Access: roles, permissions, scope

Two independent questions — *may this role do this kind of thing at all?* and *to which rows?*

Permissions, not role checks

No controller, service or repository in this codebase may contain `if (user.role === 'MANAGEMENT')`. Capabilities are named `resource:verb`; roles are bundles of them; routes declare the capability they need. At boot, the app throws if any declared permission is assigned to no role — otherwise it is dead code.

ROLE	ADDS
Employee	<code>task:read</code> , <code>task:write:own</code> , <code>task:submit</code> , <code>task:add-overtime</code> , <code>task:history:read</code> , <code>assignment:read</code> , <code>assignment:submit</code> , <code>project:read</code> , <code>team:read</code> , <code>department:read</code> , <code>dashboard:self</code> , <code>notification:read</code> , profile and password
Tech Lead	everything above, plus <code>task:write:any</code> , <code>task:approve</code> , <code>task:reject</code> , <code>task:override-lock</code> , <code>assignment:create/update/review/cancel</code> , <code>user:read</code> , <code>user:reset-password</code> , <code>dashboard:team</code> , <code>report:export</code>
Management	everything above, plus <code>task:delete</code> , full <code>user:*</code> , <code>role:assign</code> , <code>team:manage</code> , <code>project:manage</code> , <code>department:manage</code> , <code>dashboard:global</code> , <code>audit:read</code> , <code>settings:read/manage</code>

Scope: which rows

SCOPE	HELD BY	COMPILES TO
GLOBAL	Management	<code>{}</code> — the whole company
DEPARTMENT	Tech Lead with a department	<code>{ departmentId }</code>
SELF	Employee — <i>and</i> a Tech Lead with no department	<code>{ userId }</code>
unknown	—	an impossible predicate: zero rows

FAILS CLOSED

An unresolvable scope returns a sentinel no id can equal, so it matches **nothing** rather than everything. A Tech Lead with no department is downgraded to SELF — deny, never silently escalate. A bug here leaks an empty page, not another department's data.

A requested filter is merged with the permitted scope, and the permitted scope always wins. An employee asking for someone else's rows is **denied outright**, not silently rewritten to their own — silently rewriting would mask a broken client.

The UI is not the guard

The frontend hides buttons the user cannot use, using permissions the server sent. Those are cosmetic affordances. The API authorises independently and returns 403. An unauthorised page shows an explanatory lock screen inside the normal app shell — not a redirect, not a blank.

5 Module reference

Every backend module, what it is for, and the rules it enforces. The companion Flowcharts PDF has one diagram per module.

auth

Sign-in, sessions, password recovery, own profile. Access tokens are 15-minute JWTs; refresh tokens are opaque, stored only as SHA-256 hashes, and delivered in an httpOnly, secure, strict-sameSite cookie.

- **Rotation with reuse detection.** Every refresh mints a successor and revokes its predecessor. Replaying a used token revokes the entire token family and bumps the password epoch, signing the account out everywhere.
- **Token epoch.** Any access token issued before `passwordChangedAt` is rejected, so a password change, a demotion or a department move cannot be outlived by a stale token.
- **No enumeration.** Forgot-password always answers identically. Login returns one generic error, and the inactive-account check happens *after* the password check.
- **The policy is not applied at login** — applying it there tells an attacker which passwords could not exist.
- Lockout after 5 failures for 15 minutes, held in the database so it survives scale-out.

users

Employee administration, scoped so a lead sees only their department.

- You cannot change your own role or status, or delete your own account.
- The **last active Management account** cannot be demoted, deactivated or deleted.
- A Tech Lead who still leads an active team cannot be moved to another department — that would leave a team led by someone who cannot see it.
- Changing role, department or status revokes every session immediately.
- A department-scoped caller may reset **employees'** passwords only. Handing one lead the power to issue another lead a password is a lateral takeover, not a helpdesk favour.
- **Deletion stamps names onto history first**, then deletes the account — after the delete the foreign key is already NULL and there is no way left to find which rows were theirs.

departments

Departments plus their working-hour columns and bespoke task fields.

THE LOGGING CADENCE IS MANAGEMENT'S TO SET

Each department carries a **logging interval** (30 minutes to 4 hours), a day start and a day end. **Admin > Departments > Working hours** regenerates that department's grid columns from them. "Hourly" was an assumption baked into the product; it is now a setting, because the honest cadence differs by the kind of work. An editor mid-render or an engineer mid-debug cannot describe a completed unit of work every sixty minutes, and forcing them to produces "continued task" in every row — which is worse than no data, because it looks like data.

The change is two steps by design: **preview**, then apply. The preview is computed by the same planner as the write, and names every column that would be affected. `requiredSlotsPerDay` follows the new column count automatically — leaving it at 7 after halving the columns would mark every employee permanently non-compliant, and nobody would connect it to this change.

WHAT A REBUILD DOES TO EXISTING WORK

No task entry is ever deleted by a rebuild. A column the new layout has no place for is **retired** — it leaves the grid, its entries stay readable, exportable and attributable. Only genuinely empty columns are deleted. Because the database allows exactly one column per start time per department, a column whose start time survives with a different span is **re-spanned in place**: no entry moves, but the label above it changes (10:00–11:00 becomes 10:00–12:00). The preview lists those separately, with their entry counts, so that is a decision rather than a discovery. Overtime columns are never touched.

- Deletion is refused while the department holds employees, teams, real projects or logged hours (409, naming the blocker). To retire one with history, deactivate it.
- Time slots may not overlap. Removing a slot that has hours behind it **retires** it — a hard delete would cascade real work away.
- **Overtime** appends the next hour after the last column, and is excluded from required hours. An overtime column that counted toward the requirement would silently make overtime mandatory. It can be removed only while empty for everyone, and never past midnight.
- A custom field's `key` is immutable once entries carry it; change the label instead, which is what users actually see. Field deletion is soft, so old values stay queryable.

projects

- Every logged hour must name a project, so every department is seeded with an **Internal / Non-project** catch-all. It cannot be deleted and cannot leave ACTIVE — a mandatory field with no honest answer is how data goes bad.
- A project with logged hours cannot be deleted, only **archived**; archiving removes it from the picker while every hour keeps its index.
- A project cannot change department, or its hours would land on the wrong side of the isolation boundary.
- The dropdown offers ACTIVE projects only — offering an archived one is how dirty data gets created.
- **Modules** are the deliverables a project is made of. A module with assignments is retired, not deleted. `completedAt` is derived from the transition and cleared on reopen: a completion date on something still in progress is a lie the reports believe.

projects — who is on one

Until recently nobody *belonged* to a project: people belonged to departments and teams, projects belonged to departments, and the only trace of who worked on what was whoever had logged an hour. That answers a question about the past. It cannot say who is on a project before the first hour is logged, and it cannot be corrected when it is wrong.

Project membership now exists as its own record. The **Assign task** dialog reads department → project → person, and the people list puts members at the top with the hours they have logged, everyone else in the same department below them in muted text.

MUTED, NOT HIDDEN

Non-members are still selectable, deliberately. A brand-new project has no members at all, so a members-only list could never be given its first person — and putting somebody onto a project is one of the most ordinary reasons to open the dialog. Choosing one asks: *"They are in the right department but not on this project. Assign the task and add them?"* Answering yes does both, in a single transaction, so you can never end up with someone holding work on a project they are still not on.

Membership is **current state, not history**. It is the one place in this schema where deleting a person removes the row — the record of what they did lives on their task entries and assignments, which are preserved as always. Taking somebody off a project likewise removes nothing they did. Membership never crosses a department boundary.

assignments

Forward-looking work handed to a person. Hours logged against an assignment form its progress thread, and complete the chain the AI reasons over: **hour** → **assignment** → **module** → **project**.

- States: ASSIGNED → IN_PROGRESS → SUBMITTED → DONE, with reopen as the only way back.
- **Cancellation is legal only from ASSIGNED.** Once an hour has been logged, nobody — including the creator — can cancel it away: real effort is on the record.
- There is **no delete**, ever.
- Two-party handshake: only the assignee may submit; nobody may review their own work, including a Tech Lead who was assigned it.
- The assignee and department are fixed at creation and taken from the assignee, never from the client. The project is immutable; only the module may be re-pointed, and only within the same project.
- A percentage complete is shown only where an `estimatedHours` estimate exists. Otherwise the UI shows effort and never a guessed percentage.
- The first hour logged flips ASSIGNED → IN_PROGRESS inside the same transaction that saves the entry.

tasks

The hourly grid and the day approval workflow. Five invariants: scope, locking, concurrency, history, backdating.

- **Day states:** DRAFT → SUBMITTED → APPROVED or REJECTED. Editing a rejected sheet returns it to DRAFT automatically. An approved day can be reopened only by a lead, as an explicit audited override.
- **Two things lock a day:** its status (SUBMITTED/APPROVED), and age beyond the backdating window (default 2 days). Future dates are refused outright.

- **Lateness is measured, not blocked.** `isLate` is computed server-side from the slot's end plus grace — never trusted from the client, never inferred from `updatedAt`.
- **Submission requires completeness**, counted against the real current slots rather than the configured integer, so an admin's grid change cannot demand a column that no longer exists.
- **Autosave never creates an unprojected row.** It is skipped and reported as skipped, so the cell stays honestly "unsaved" — which is exactly what is true.
- **Every mutation writes a revision** in the same transaction, storing the project *name* as well as its id so the history stays readable in two years after a rename. A delete writes its tombstone before the row goes.
- Optimistic concurrency: a stale `version` returns 409 with the server's current row attached, so the UI can offer "use theirs / keep mine" instead of losing the edit.
- Break slots are not writable. Overtime slots are — that is the entire point of them.

reports

- Excel, CSV or PDF, streamed, always scoped to the caller — a lead's export contains only their department.
- Every export writes a `REPORT_EXPORTED` audit row. Exporting data is an event a compliance officer will one day want to see.
- CSV is escaped against formula injection.

dashboard

- The compliance denominator counts only hours that have actually **finished**, excluding the in-progress hour, breaks and overtime. A metric that is red by construction every morning is one everybody ignores by lunchtime.
- Team follow-up separates fill rate from on-time rate: a team at 100% fill and 30% on-time is **backfilling**, not complying.
- Departments with zero activity are kept on the chart. A department that logged nothing is the most important bar on it.

notifications

- You can only read and mark your own — there is no `userId` parameter anywhere in this module.
- Persist first, email second. The bell is the system of record; email is best effort.
- De-duplication is a UNIQUE index in the database, not an application check that races itself the moment there is more than one instance. Only the rows that actually landed are emailed.
- Deactivated accounts are never mailed — that address may have been reassigned.

audit & settings

- The audit log is **append-only**: no update or delete endpoint exists, and the retention job explicitly spares it. The record of who deleted the data must outlive the data.
- Audit writes never break a user action — they are fire-and-forget and loudly logged on failure, except where atomicity matters (role changes, approvals, deletes), where they join the transaction.
- Sensitive keys are **dropped entirely, not masked**, from audit diffs.
- Settings accepts only known keys — an open key-value store just ships typos.

6 The refusals

The behaviours most likely to surprise a new administrator. Each is deliberate.

YOU TRY TO...	WHAT HAPPENS	WHY
Delete a department with any history	409, naming the blocker	Its people, projects and hours would be orphaned. Deactivate instead.
Delete a team that still has members	409	They would be silently orphaned with nothing to explain why.
Delete a project with logged hours	Refused — archive instead	Every hour points at a project; it is the index all reporting reads.
Archive the Internal / Non-project project	Refused	Hours that belong to no project would have nowhere honest to go.
Remove a working hour that has entries	Retired, not destroyed	A hard delete would cascade real work away.
Delete an employee	Allowed — the work stays	The login goes; the hours remain, attributed by name.
Cancel an assignment already worked on	Refused	Real effort has been recorded against it.
Delete an assignment	No such operation	Cancellation preserves the trail of who assigned what.
Approve your own sheet or assignment	403	A lead who is assigned work must not sign it off.
Demote the last Management account	Refused	Nobody would be left who could undo it.
Change a custom field's key after use	Refused	Existing entries carry that key. Change the label.
Read AI findings as an employee	403, always	They are notified, never shown. See §7.
Log an hour for a future date	Refused	Nothing has happened yet.
Log an hour late	Allowed , flagged LATE	Measure the behaviour; do not prevent the record.
Rebuild a grid over logged work	Columns are retired or re-spanned, never deleted	Changing the cadence must not cost anybody the hours they recorded.
Add someone from another department to a project	400 PROJECT_DEPARTMENT_MISMATCH	A project belongs to one department; membership cannot cross the isolation boundary.
Assign work to a non-member	Allowed , after a confirmation	It is how people join projects. It is a second decision, so it is asked, not assumed.
Review a department that has opted out	400 DEPARTMENT_OPTED_OUT	The opt-out means "not on request" as well as "not on a schedule".
Narrow a review to a project or a person from another department	400 PROJECT_DEPARTMENT_MISMATCH / USER_DEPARTMENT_MISMATCH	Silently matching nobody would return "0 flagged" — a clean bill of health for a question never actually asked.
Set the AI cadence to 0 or 99 hours	400 INVALID_SETTING_VALUE	A known key is not the same as a sane value.

ONE TRAP THAT IS NOT A REFUSAL

`PATCH /departments/:id` is **not** a partial update. Fields omitted from the payload are blanked. The admin drawer always sends the whole object, so the UI is safe; a hand-rolled API call is not.

What survives what

DELETING...	DESTROYS	PRESERVES
An employee	Refresh tokens, reset OTPs, notifications	Task days, entries, revisions, assignments, transitions, audit rows, rollups, AI findings — all still named
A team	The team row	Its members' accounts and every hour they logged
An assignment (cancel)	Nothing	Every hour logged against it
A project module (retire)	Nothing	Its assignments and their hours

Twenty relations use `SetNull` plus a name snapshot to hold this contract; only three cascade, and all three are personal credentials.

7 The AI analyser

On the cadence Management sets — two hours by default — per person: do the hours logged plausibly advance the work assigned?

What it does

For each candidate it builds one evidence object — their open assignments, and the hour descriptions they wrote in the window — and asks Gemini for a structured verdict: a **kind** (MISALIGNED, IDLE, LOW_SUBSTANCE, AT_RISK, ON_TRACK), a **severity**, an optional 0–100 alignment score, a finding and a recommendation.

The four rules it enforces

1. **Write the finding before telling anyone.** A model's judgement about a person's work is an accusation with consequences. It lands in a durable row — with the evidence it saw and the model that said it — so a disputed finding can be audited against its inputs instead of re-argued from memory.
2. **The employee is notified, not shown.** Leads and Management get the finding. The employee gets a neutral, actionable nudge — never the reasoning, never a score. They can fix the record; they are not handed a machine's opinion of them. `GET /ai/insights` returns 403 for an employee, enforced in the service as well as the route.
3. **Evidence or silence.** No assignments and nothing logged means nothing to judge. Absence of data is not evidence of idleness, and inventing a finding from it is how a tool like this loses its users' trust in a week.
4. **It never blocks anything.** No timesheet is rejected, no account touched. Every failure path — no key, no quota, bad JSON — degrades to "no insights this run".

ON_TRACK findings are stored too. An insight table that only ever records problems reads as a charge sheet, and makes it impossible to tell "the AI checked and was satisfied" from "the AI never looked".

Per-department opt-out

ENFORCED IN THE SELECT, NOT AS A FILTER ON THE RESULTS

`Department.aiAnalysisEnabled` is part of the analyser's **candidate query**. A department that has switched it off never becomes a candidate, so its assignments and hour descriptions are never read out of the database and nothing about it leaves the network. Analysing everyone and hiding the findings afterwards would mean the work had already left by the time we honoured the setting. That is not an opt-out.

The toggle is Management-only and audited. `/ai/status` reports which departments have opted out, so "nothing flagged" is never ambiguous.

What leaves the network

For an opted-in department only: employee name, code and department; the analysis window; their open assignments (title, description, project, module, status, priority, due date, estimate); and the hour descriptions they wrote, with the project and assignment each was logged against. No email addresses, no credentials, no salary or HR data — the system holds none.

Cost and reliability

DECISION	BASIS
Model <code>gemin</code> <code>2.5-flash</code>	Measured 8/8 successful, median 1.2s. <code>gemin</code> <code>3.5-flash</code> managed 3/8 (five 503s), median 8.7s. A model that fails 60% of the time judges nobody.
One call per employee	Batching people into one call invites cross-attribution. In a system that accuses people, that costs more than tokens.
Compact JSON	Pretty-printing cost 243 of ~1,028 tokens per call — a quarter of every request spent on indentation the model does not read.
Skip unchanged evidence	A fingerprint of the substantive evidence, excluding window timestamps. If today's last finding came from identical evidence, nothing is sent. Bounded to the work date, so everyone is still genuinely assessed at least once a day.
Working hours only	The job wakes hourly on 10 9–19 * * 1–5. A 03:00 run cannot find anything a 09:00 run will not; all it produces is a metered call and an IDLE finding about somebody's night.
Cadence set by Management	<code>ai.analysis.intervalHours</code> in Admin > Settings. The job wakes hourly and works only when that many hours have passed, so the change takes effect immediately with no redeploy. Raising it from 2 to 4 halves the calls. The window always matches the cadence, so nothing goes unexamined.

The period review — the on-demand check

The scheduled analyser answers *"is today going wrong?"*. It cannot answer *"has this person moved in a fortnight?"*, because its window is two hours long. The **period review** does: Management picks a department and a period (7–30 days) on the AI Insights screen and presses **Run AI detection**.

Two further dropdowns — **Project** and **Employee** — narrow the question. Both are optional and both default to everything, so the button pressed as it stands is the whole-department review it has always been. A project gathers everyone connected to it three ways: on its member list, holding an assignment on it, or having logged an hour against it in the period. Membership alone would miss whoever is quietly helping out; hours alone would miss the person assigned to it who has not started, who is the more interesting of the two. Naming an employee overrides both — an explicit instruction is not a filter to be second-guessed.

Each active person in that department is read across the whole period at once — their daily log, their assignments, and crucially the **status and completion date of the project modules** that work belongs to. That last part is what turns a suspicion into a finding: hours logged against a module the project itself records as finished is a contradiction, not a similarity score.

WHAT ONLY A REVIEW CAN CATCH

Three entries — *"tested communication module"*, *"testing schoolmate communication module"*, *"testing communication module of schoolmate app"* — spread across a fortnight. Three different strings, so exact matching finds nothing, and each one read alone is unremarkable. Laid side by side against a module completed on day five, it is a finding: `NO_PROGRESS`.

THREE GUARDS THAT KEEP IT HONEST

Repetition alone is never a finding. Testing one large module for three weeks is normal engineering.

`NO_PROGRESS` requires restatement *plus* hard evidence — the module already complete, or effort past its estimate with nothing moving. A completed module alone is not enough either: bug fixes and follow-ups legitimately land after sign-off, so distinct advancing entries after completion read as `ON_TRACK`.

Absence of data can never be CRITICAL. This system holds no record of leave, sick days or onboarding, so a quiet fortnight is explained equally well by any of them. The cap is enforced in code, not in the prompt — the prompt already said it and the model returned CRITICAL anyway.

A project filter must not libel anyone. Narrowed to one project, the model sees that project's share of a person's time and nothing else — so somebody who spent the fortnight heads-down on a different project shows up with an empty log, which reads exactly like idleness. Withholding their other hours is right; withholding the *fact* of them is what turns a filter into an accusation. The evidence therefore carries one integer, `coverage.entriesElsewhere`, counting the hours held back. Two identical two-hour Schoolmate logs, one with 0 elsewhere and one with 14, correctly return `NO_PROGRESS` and `ON_TRACK`. Severity is capped at INFO in code for anyone who logged nothing here but plenty elsewhere: that is a fact about allocation, not about effort.

Nobody is notified by a review. The scheduled analyser alerts leads because it catches things worth correcting the same day; a review is retrospective, so the result goes to the person who asked and no further. Reviews are listed on their own tab, never mixed with the live feed — a fortnight-long judgement must not read as something that happened this afternoon. A project-scoped verdict is stored under its own key, so it sits *beside* the whole-period one rather than overwriting it; naming an employee does not, because reviewing one person over a period already reviewed is the same question asked again and should refresh the answer, not duplicate it.

Turning it off

Three levers, coarsest first: unset `GEMINI_API_KEY` (dormant); set `AI_ANALYSIS_ENABLED=false` (configured but off); or switch off individual departments in their admin drawer.

8 Scheduled jobs

Nine jobs, any number of instances, exactly one run each.

JOB	WHEN	WHAT IT DOES
retention-cleanup	00:05	Deletes task history older than 180 days, children first, in batches of 1,000. Never touches users, projects, settings or the audit log.
productivity-rollup	00:20	Rebuilds yesterday and today per person. Idempotent by construction.
unsubmitted-check	09:15, Mon–Fri	Nudges anyone whose previous day is still a draft.
assignment-reminders	09:30, Mon–Fri	Due-soon and overdue nudges; overdue rolls up to whoever assigned it.
ai-analysis	Wakes :10 hourly 09–19, Mon–Fri; works on the cadence Management sets (default 2h)	The work-alignment analyser (\$7).
auto-approval	:40 hourly	Approves sheets submitted more than 24 hours ago that nobody reviewed. Recorded as the system, not a person, and still reopenable.
hourly-reminders	:50 hourly	Three-way escalation for overdue hours: employee, then their lead, then Management. Skips non-working days and holidays.
lead-digest	13:30 & 18:30, Mon–Fri	Emails each lead who on their team is behind.
management-summary	19:00, Mon–Fri	Company-wide daily summary.

All times are `CRON_TIMEZONE`, default `Asia/Kolkata`. Every schedule is overridable by environment variable, and every expression is validated at boot — an invalid one is refused loudly, because nobody notices a silently unscheduled retention job until the database is 400 GB.

WHY THE LOCKING MATTERS

node-cron runs inside the application process, and App Service scales out. With three instances every expression fires three times: three reminder emails to each employee, and three concurrent 180-day delete transactions racing on the same rows. Each job takes a lease on a database row via a single atomic conditional update — either it affects one row (we won) or none (someone else holds it). There is no check-then-act window. Leases expire, so an instance killed mid-run does not hold the lock forever.

A job that throws is swallowed and logged. An unhandled rejection inside a cron tick would kill the API process that people are using to log their hours. Job status and a "run now" button live in Admin > Settings.

9 Security posture

AREA	POSITION
Passwords	bcrypt, ≥12 rounds; minimum length plus upper, lower, digit and symbol; the policy is not applied at login
Access tokens	15-minute JWT, algorithm pinned — without pinning, a token claiming <code>alg: none</code> is accepted
Refresh tokens	Opaque, hashed at rest, rotated on use, family-revoked on replay (RFC 6819 §5.2.2.3)
In the browser	The access token lives in a module variable only — never localStorage, so XSS cannot read it. The refresh token is an httpOnly cookie the JS never sees.
Every request	The user is re-read from the database. A stateless token cannot know the account was deactivated ninety seconds ago.
Pagination	Hard page-size ceiling (<code>?pageSize=1000000</code> is a trivial DoS) and a sort-field allow-list — passing <code>sortBy</code> straight through lets a caller sort by <code>passwordHash</code> , which with pagination is a practical oracle for extracting hashes.
Errors	One exit point. Prisma errors are translated, never leaked — a raw Prisma error exposes table and column names. Unexpected errors return only a correlation ID.
Uploads	Avatars verified by magic bytes, not by file extension
Audit	Append-only; sensitive fields dropped from diffs entirely rather than masked

10 Operations

Deploying

`DEPLOYMENT.md` at the repo root carries the full procedure; `docker-compose.yml`, `azure-setup.sh` and `azure-deploy.sh` are the scaffolding. In production:

- Use the restricted application user from `db-setup.sql`, not root.
- Set `TRUST_PROXY` to the number of proxies in front of the app. Get it wrong and every audit row records the load balancer's IP, and every IP rate limit becomes one shared bucket for the whole internet.
- Watch `connection_limit`: a pool per instance adds up fast against a small MySQL tier.
- Run migrations with `db:deploy`. Never `db:reset` — it drops everything.

Backups and retention

Task history is deleted after `TASK_RETENTION_DAYS` (180). That is a business policy, so it is configurable — but it is a real delete. Confirm your database backup covers the retention horizon before going live, because after the sweep the only remaining record of a deleted period is the audit log.

Testing

SUITE	COMMAND
Unit	<code>npm run test:unit</code>
Integration	<code>node --env-file=.env tests/integration/api.smoke.mjs</code>
Data preservation	<code>node --env-file=.env tests/integration/data-preservation.mjs</code>

RUN ONE INTEGRATION SUITE PER FRESH DATABASE

These suites mutate shared state and are **not idempotent**. `data-preservation.mjs` deliberately deletes a seeded employee — that is the point of it — and `api.smoke.mjs` signs in as that same person. Running them back to back produces failures that look like product bugs and are not. Reset, restart the API, run one suite. Isolated, both are green.

Start the API with `node --env-file=.env src/server.js` for test runs, not `npm run dev`: the dev script uses `--watch`, which restarts mid-suite and shows up as `ECONNRESET`. Both suites honour `SMOKE_ADMIN_EMAIL` and `SMOKE_ADMIN_PASSWORD`.

Troubleshooting

SYMPTOM	CAUSE
404 on <code>/api/health</code>	Liveness is <code>/healthz</code> ; the API prefix is <code>/api/v1</code>
API will not start	Config validation. The first lines of output name the offending variable.
EPERM on <code>prisma generate</code>	A running Node process holds the query-engine DLL. Stop the server first.
Everything returns an empty list	A scope that could not resolve — it fails closed. Check the user's department.
409 on save	Optimistic concurrency: someone else changed that row. The response carries their version.
AI reports 0 assessed	Check <code>/ai/status</code> for opted-out departments, and whether evidence was unchanged — the run log distinguishes them.

11 Known gaps

- **Rotate the credentials used during development.** The Gemini API key and the Gmail app password were both exposed in working sessions and should be reissued before go-live. Add `SMTP_REQUIRE_TLS=true`.
- Reuse of one password across MySQL and the application's admin account should be separated.
- The AI's `aiSummary` / `aiRiskScore` fields on assignments are dormant seams, not features.
- Notifications are polled (30–60s), not pushed. Fine at this scale; the seam for websockets is clean if needed.

COMPANION DOCUMENT

Ara-Workbench-Flowcharts.pdf — 24 diagrams, one per module, covering the request lifecycle, access scope, every business module, the logging cadence, the scheduled jobs, and the data-preservation contract drawn out in full.